컴퓨터 구조 Project Report

학과	전자정보공학부(IT 융합전공)
학번	20201599
이름	이건아

우선 성능 분석을 하기 전 이해를 돕기 위해 Simulation Statistics에 대한 각각의 정의에 대해 알아보았습니다.

이름	정의
sim_num_insn	총 실행된 명령어 수
sim_num_refs	총 실행된 로드와 스토어 수
sim_num_loads	총 실행된 로드 수
sim_num_stores	총 실행된 스토어 수
sim_num_branches	총 실행된 분기 수
sim_elapsed_time	총 시뮬레이션 시간(초 단위로 나타냄)
sim_inst_rate	시뮬레이션 속도 (명령어/초)
sim_total_insn	총 실행된 명령어 수
sim_total_refs	총 실행된 로드와 스토어 수
sim_total_loads	총 실행된 로드 수
sim_total_stores	총 실행된 스토어 수
sim_total_branches	총 실행된 분기 수
sim_cycle	시뮬레이션에 걸린 총 사이클 수
sim_IPC	Instructions Per Cycle (IPC)로, 한 사이클에 실행된 명령어 수의 평균
sim_CPI	Cycles Per Instruction (CPI)로, 한 명령어를 실행하는 데 걸린 사이클의 평균
sim_exec_BW	한 사이클에 실행되는 총 명령어 수 (오류 및 실행 완료)
sim_IPB	분기당 명령어 수
IFQ_count	누적된 IFQ(Instruction Fetch Queue) 착용 수
IFQ_fcount	누적된 IFQ 가득 찬 횟수
ifq_occupancy	평균 IFQ 착용 수 (명령어)
ifq_rate	평균 IFQ 디스패치 비율 (명령어/사이클)
ifq_latency	평균 IFQ 착용 지연 시간 (사이클)
ifq_full	IFQ가 가득 찬 시간의 비율 (사이클)
RUU_count	누적된 RUU(Register Update Unit) 착용 수
RUU_fcount	누적된 RUU 가득 찬 횟수
ruu_occupancy	평균 RUU 착용 수 (명령어)
ruu_rate	평균 RUU 디스패치 비율 (명령어/사이클)
ruu_latency	평균 RUU 착용 지연 시간 (사이클)
ruu_full	RUU가 가득 찬 시간의 비율 (사이클)
LSQ_count	누적된 LSQ(Load Store Queue) 착용 수
LSQ_fcount	누적된 LSQ 가득 찬 횟수
lsq_occupancy	평균 LSQ 착용 수 (명령어)
lsq_rate	평균 LSQ 디스패치 비율 (명령어/사이클)
lsq_latency	평균 LSQ 착용 지연 시간 (사이클)
lsq_full	LSQ가 가득 찬 시간의 비율 (사이클)
sim_slip	총 슬립 사이클 수
avg_sim_slip	발행과 퇴직 사이의 평균 슬립
bpred_bimod.lookups	총 분기 예측 조회 수
bpred_bimod.updates	총 분기 예측 업데이트 수
bpred_bimod.addr_hits	총 주소 예측 히트 수
bpred_bimod.dir_hits	총 방향 예측 히트 수 (주소 히트 포함)
bpred_bimod.misses	총 예측 미스 수
bpred_bimod.jr_hits	JR 주소 예측 히트 수

bpred_bimod.jr_seen	본 적 있는 JR 수
bpred_bimod.jr_non_ras_hits.PP	주소 예측 히트 수 (비 RAS JR)
bpred_bimod.jr_non_ras_seen.PP	본 적 있는 비 RAS JR 수
bpred_bimod.bpred_addr_rate	분기 주소 예측률 (예: addr-hits/updates)
bpred_bimod.bpred_dir_rate	분기 방향 예측률 (예: all-hits/updates)
bpred_bimod.bpred_jr_rate	JR 주소 예측률 (예: JR addr-hits/JRs seen)
bpred_bimod.bpred_jr_non_ras_rate.PP	비 RAS JR 주소 예측률 (예: 비 RAS JR 히트 수/JRs seen)
bpred_bimod.retstack_pushes	반환 주소 스택에 push된 주소 수
bpred_bimod.retstack_pops	반환 주소 스택에서 pop된 주소 수
bpred_bimod.used_ras.PP	사용된 RAS 예측 수
bpred_bimod.ras_hits.PP	RAS 히트 수
bpred_bimod.ras_rate.PP	RAS 예측률 (예: RAS 히트 수/사용된 RAS)
il1.accesses	총 il1 캐시의 엑세스 수
il1.hits	il1 캐시의 히트 수
il1.misses	il1 캐시의 미스 수
il1.replacements	il1 캐시의 교체 수
il1.writebacks	il1 캐시의 쓰기 수
il1.invalidations	il1 캐시의 무효화 수
il1.miss_rate	il1 미스 비율 (예: 미스/참조)
il1.repl_rate	il1 캐시의 교체율, 참조당 교체 횟수
il1.wb_rate	il1 캐시의 쓰기 회귀율, 참조당 writeback의 횟수
il1.inv_rate	il1 캐시의 무효화율, 참조당 무효화 횟수
dl1.accesses	총 dl1 캐시의 엑세스 수
dl1.hits	dl1 캐시의 히트 수
dl1.misses	dl1 캐시의 미스 수
dl1.replacements	dl1 캐시의 교체 수
dl1.writebacks	dl1 캐시의 쓰기 수
dl1.invalidations	dl1 캐시의 무효화 수
dl1.miss_rate	dl1 미스 비율 (예: 미스/참조)
dl1.repl_rate	dl1 캐시의 교체율, 참조당 교체 횟수
dl1.wb_rate	dl1 캐시의 쓰기 회귀율, 참조당 writeback의 횟수
dl1.inv_rate	dl1 캐시의 무효화율, 참조당 무효화 횟수
ul2.accesses	총 ul2 캐시의 엑세스 수
ul2.hits	ul2 캐시의 히트 수
ul2.misses	ul2 캐시의 미스 수
ul2.replacements	ul2 캐시의 교체 수
ul2.writebacks	ul2 캐시의 쓰기 수
ul2.invalidations	ul2 캐시의 무효화 수
ul2.miss_rate	ul2 미스 비율 (예: 미스/참조)
ul2.repl_rate	ul2 캐시의 교체율, 참조당 교체 횟수
ul2.wb_rate	ul2 캐시의 쓰기 회귀율, 참조당 writeback의 횟수
ul2.inv_rate	ul2 캐시의 무효화율, 참조당 무효화 횟수
itlb.accesses	총 itlb 캐시의 엑세스 수
itlb.hits	itlb 캐시의 히트 수
itlb.misses	itlb 캐시의 미스 수
itlb.replacements	itlb 캐시의 교체 수
itlb.writebacks	itlb 캐시의 쓰기 수

itlb.invalidations	itlb 캐시의 무효화 수
itlb.miss_rate	itlb 미스 비율 (예: 미스/참조)
itlb.repl_rate	itlb 캐시의 교체율, 참조당 교체 횟수
itlb.wb_rate	itlb 캐시의 쓰기 회귀율, 참조당 writeback의 횟수
itlb.inv_rate	itlb 캐시의 무효화율, 참조당 무효화 횟수
dtlb.accesses	총 dtlb 캐시의 액세스 수
dtlb.hits	dtlb 캐시의 히트 수
dtlb.misses	dtlb 캐시의 미스 수
dtlb.replacements	dtlb 캐시의 교체 수
dtlb.writebacks	dtlb 캐시의 쓰기 수
dtlb.invalidations	dtlb 캐시의 무효화 수
dtlb.miss_rate	dtlb 미스 비율 (예: 미스/참조)
dtlb.repl_rate	dtlb 캐시의 교체율, 참조당 교체 횟수
dtlb.wb_rate	dtlb 캐시의 쓰기 회귀율, 참조당 writeback의 횟수
dtlb.inv_rate	dtlb 캐시의 무효화율, 참조당 무효화 횟수
sim_invalid_addrs	명령어 실행 중에 발생한 정상적이지 않은 주소의 총 수
ld_text_base	프로그램 코드 세그먼트의 시작 주소
ld_text_size	바이트 단위의 프로그램 코드 세그먼트의 크기
ld_data_base	프로그램 초기화된 데이터 세그먼트의 기준 주소
ld_data_size	.data 및 초기화되지 않은 .bss 세그먼트의 크기를 바이트 단위로 나타낸다.
ld_stack_base	프로그램 스택 세그먼트의 기준 주소
ld_stack_size	프로그램 스택의 초기 크기
ld_prog_entry	프로그램 실행이 시작되는 메모리 주소
ld_environ_base	프로그램 환경의 기준 주소
ld_target_big_endian	대상 실행 파일이 big_endian인 경우 0이 아닌 값이며, 그렇지 않으면 0이다.
mem.page_count	할당된 전체 페이지 수
mem.page_mem	할당된 메모리 페이지의 총 크기를 킬로바이트 단위로 나타낸다.
mem.ptab_misses	첫 번째 수준 페이지 테이블에서 발생한 미스 횟수를 추적
mem.ptab_accesses	페이지 테이블 전체 액세스 횟수
mem.ptab_miss_rate	첫 번째 수준 페이지 테이블 미스 비율로, 전체 페이지 테이블 액세스 중에서 발생한 미스의 비율

- 각 Benchmark Application에 대해 Baseline.cfg로 측정한 성능 분석

먼저 Gcc에 대해 Baseline.cfg로 측정한 성능은 다음과 같습니다.

```
sim: ** simulation statistics **
sim_num_insn              337353533 # total number of instructions committed
sim_num_refs              121902601 # total number of loads and stores committed
sim_num_loads              83209483 # total number of loads committed
sim_num_stores         38693118.0000 # total number of stores committed
sim_num_branches           58874955 # total number of branches committed
sim_elapsed_time                183 # total simulation time in seconds
sim_inst_rate          1843461.9290 # simulation speed (in insts/sec)
sim_total_insn            395906021 # total number of instructions executed
sim_total_refs            143451509 # total number of loads and stores executed
sim_total_loads           100718267 # total number of loads executed
sim_total_stores       42733242.0000 # total number of stores executed
sim_total_branches         68762165 # total number of branches executed
sim_cycle                 273328572 # total simulation time in cycles
sim_IPC                      1.2342 # instructions per cycle
sim_CPI                      0.8102 # cycles per instruction
sim_exec_BW                  1.4485 # total instructions (mis-spec + committed) per cycle
sim_IPB                      5.7300 # instruction per branch
IFQ_count                 621753694 # cumulative IFQ occupancy
IFQ_fcount                137610942 # cumulative IFQ full count
ifq_occupancy                2.2747 # avg IFQ occupancy (insn's)
ifq_rate                     1.4485 # avg IFQ dispatch rate (insn/cycle)
ifq_latency                  1.5705 # avg IFQ occupant latency (cycle's)
ifq_full                     0.5035 # fraction of time (cycle's) IFQ was full
RUU_count                2301295262 # cumulative RUU occupancy
RUU_fcount                 66057847 # cumulative RUU full count
ruu_occupancy                8.4195 # avg RUU occupancy (insn's)
ruu_rate                     1.4485 # avg RUU dispatch rate (insn/cycle)
ruu_latency                  5.8127 # avg RUU occupant latency (cycle's)
ruu_full                     0.2417 # fraction of time (cycle's) RUU was full
LSQ_count                 836129874 # cumulative LSQ occupancy
LSQ_fcount                 30794261 # cumulative LSQ full count
lsq_occupancy                3.0591 # avg LSQ occupancy (insn's)
lsq_rate                     1.4485 # avg LSQ dispatch rate (insn/cycle)
lsq_latency                  2.1119 # avg LSQ occupant latency (cycle's)
lsq_full                     0.1127 # fraction of time (cycle's) LSQ was full
sim_slip                  3272483020 # total number of slip cycles
avg_sim_slip                 9.7005 # the average slip between issue and retirement
bpred_bimod.lookups        72315779 # total number of bpred lookups
bpred_bimod.updates        58874955 # total number of updates
bpred_bimod.addr_hits      50991123 # total number of address-predicted hits
bpred_bimod.dir_hits       52583992 # total number of direction-predicted hits (includes addr-hits)
bpred_bimod.misses          6290963 # total number of misses
bpred_bimod.jr_hits         4975049 # total number of address-predicted hits for JR's
bpred_bimod.jr_seen         6304002 # total number of JR's seen
bpred_bimod.jr_non_ras_hits.PP       534855 # total number of address-predicted hits for non-RAS JR's
bpred_bimod.jr_non_ras_seen.PP      1713371 # total number of non-RAS JR's seen
bpred_bimod.bpred_addr_rate    0.8661 # branch address-prediction rate (i.e., addr-hits/updates)
```

```
bpred_bimod.bpred_dir_rate    0.8931 # branch direction-prediction rate (i.e., all-hits/updates)
bpred_bimod.bpred_jr_rate    0.7892 # JR address-prediction rate (i.e., JR addr-hits/JRs seen)
bpred_bimod.bpred_jr_non_ras_rate.PP    0.3122 # non-RAS JR addr-pred rate (ie, non-RAS JR hits/JRs seen)
bpred_bimod.retstack_pushes      5889920 # total number of address pushed onto ret-addr stack
bpred_bimod.retstack_pops      5596234 # total number of address popped off of ret-addr stack
bpred_bimod.used_ras.PP      4590631 # total number of RAS predictions used
bpred_bimod.ras_hits.PP      4440194 # total number of RAS hits
bpred_bimod.ras_rate.PP    0.9672 # RAS prediction rate (i.e., RAS hits/used RAS)
il1.accesses               427424004 # total number of accesses
il1.hits                   415597859 # total number of hits
il1.misses                  11826145 # total number of misses
il1.replacements            11825633 # total number of replacements
il1.writebacks                     0 # total number of writebacks
il1.invalidations                  0 # total number of invalidations
il1.miss_rate                 0.0277 # miss rate (i.e., misses/ref)
il1.repl_rate                 0.0277 # replacement rate (i.e., repls/ref)
il1.wb_rate                   0.0000 # writeback rate (i.e., wrbks/ref)
il1.inv_rate                  0.0000 # invalidation rate (i.e., invs/ref)
dl1.accesses               128845258 # total number of accesses
dl1.hits                   125378651 # total number of hits
dl1.misses                   3466607 # total number of misses
dl1.replacements             3466095 # total number of replacements
dl1.writebacks                902194 # total number of writebacks
dl1.invalidations                  0 # total number of invalidations
dl1.miss_rate                 0.0269 # miss rate (i.e., misses/ref)
dl1.repl_rate                 0.0269 # replacement rate (i.e., repls/ref)
dl1.wb_rate                   0.0070 # writeback rate (i.e., wrbks/ref)
dl1.inv_rate                  0.0000 # invalidation rate (i.e., invs/ref)
ul2.accesses                16194946 # total number of accesses
ul2.hits                    15278731 # total number of hits
ul2.misses                    916215 # total number of misses
ul2.replacements              912119 # total number of replacements
ul2.writebacks                208143 # total number of writebacks
ul2.invalidations                  0 # total number of invalidations
ul2.miss_rate                 0.0566 # miss rate (i.e., misses/ref)
ul2.repl_rate                 0.0563 # replacement rate (i.e., repls/ref)
ul2.wb_rate                   0.0129 # writeback rate (i.e., wrbks/ref)
ul2.inv_rate                  0.0000 # invalidation rate (i.e., invs/ref)
itlb.accesses              427424004 # total number of accesses
itlb.hits                  427307307 # total number of hits
itlb.misses                   116697 # total number of misses
itlb.replacements             116633 # total number of replacements
itlb.writebacks                    0 # total number of writebacks
itlb.invalidations                 0 # total number of invalidations
itlb.miss_rate                0.0003 # miss rate (i.e., misses/ref)
```

```
itlb.repl_rate                0.0003 # replacement rate (i.e., repls/ref)
itlb.wb_rate                  0.0000 # writeback rate (i.e., wrbks/ref)
itlb.inv_rate                 0.0000 # invalidation rate (i.e., invs/ref)
dtlb.accesses              129869552 # total number of accesses
dtlb.hits                  129822551 # total number of hits
dtlb.misses                   47001 # total number of misses
dtlb.replacements             46873 # total number of replacements
dtlb.writebacks                   0 # total number of writebacks
dtlb.invalidations                0 # total number of invalidations
dtlb.miss_rate                0.0004 # miss rate (i.e., misses/ref)
dtlb.repl_rate                0.0004 # replacement rate (i.e., repls/ref)
dtlb.wb_rate                  0.0000 # writeback rate (i.e., wrbks/ref)
dtlb.inv_rate                 0.0000 # invalidation rate (i.e., invs/ref)
sim_invalid_addrs                 0 # total non-speculative bogus addresses seen (debug var)
ld_text_base           0x0120000000 # program text (code) segment base
ld_text_size                1564672 # program text (code) size in bytes
ld_data_base           0x0140000000 # program initialized data segment base
ld_data_size                 277104 # program init'ed `.data' and uninit'ed `.bss' size in bytes
ld_stack_base          0x011ff9b000 # program stack segment base (highest address in stack)
ld_stack_size                16384 # program initial stack size
ld_prog_entry          0x0120025f70 # program entry point (initial PC)
ld_environ_base        0x011ff97000 # program environment base address address
ld_target_big_endian              0 # target executable endian-ness, non-zero if big endian
mem.page_count                  655 # total number of pages allocated
mem.page_mem                   5240k # total size of memory pages allocated
mem.ptab_misses               950796 # total first level page table misses
mem.ptab_accesses         2661620259 # total page table accesses
mem.ptab_miss_rate            0.0004 # first level page table miss rate
```

Anagram에 대해 Baseline.cfg로 측정한 성능은 다음과 같습니다.

```
sim: ** simulation statistics **
sim_num_insn               25597522 # total number of instructions committed
sim_num_refs                9033559 # total number of loads and stores committed
sim_num_loads               6491588 # total number of loads committed
sim_num_stores         2541971.0000 # total number of stores committed
sim_num_branches            3778362 # total number of branches committed
sim_elapsed_time                 10 # total simulation time in seconds
sim_inst_rate          2559752.2000 # simulation speed (in insts/sec)
sim_total_insn             27004989 # total number of instructions executed
sim_total_refs              9476311 # total number of loads and stores executed
sim_total_loads             6795163 # total number of loads executed
sim_total_stores       2681148.0000 # total number of stores executed
sim_total_branches          4006787 # total number of branches executed
sim_cycle                  11703806 # total simulation time in cycles
sim_IPC                       2.1871 # instructions per cycle
sim_CPI                       0.4572 # cycles per instruction
sim_exec_BW                   2.3074 # total instructions (mis-spec + committed) per cycle
sim_IPB                       6.7748 # instruction per branch
IFQ_count                  41780680 # cumulative IFQ occupancy
IFQ_fcount                  9437819 # cumulative IFQ full count
ifq_occupancy                 3.5698 # avg IFQ occupancy (insn's)
ifq_rate                      2.3074 # avg IFQ dispatch rate (insn/cycle)
ifq_latency                   1.5471 # avg IFQ occupant latency (cycle's)
ifq_full                      0.8064 # fraction of time (cycle's) IFQ was full
RUU_count                 160355430 # cumulative RUU occupancy
RUU_fcount                  6898572 # cumulative RUU full count
ruu_occupancy                13.7011 # avg RUU occupancy (insn's)
ruu_rate                      2.3074 # avg RUU dispatch rate (insn/cycle)
ruu_latency                   5.9380 # avg RUU occupant latency (cycle's)
ruu_full                      0.5894 # fraction of time (cycle's) RUU was full
LSQ_count                  54506253 # cumulative LSQ occupancy
LSQ_fcount                  1202059 # cumulative LSQ full count
lsq_occupancy                 4.6571 # avg LSQ occupancy (insn's)
lsq_rate                      2.3074 # avg LSQ dispatch rate (insn/cycle)
lsq_latency                   2.0184 # avg LSQ occupant latency (cycle's)
lsq_full                      0.1027 # fraction of time (cycle's) LSQ was full
sim_slip                  245007984 # total number of slip cycles
avg_sim_slip                  9.5716 # the average slip between issue and retirement
bpred_bimod.lookups         4101685 # total number of bpred lookups
bpred_bimod.updates         3778362 # total number of updates
bpred_bimod.addr_hits       3632135 # total number of address-predicted hits
bpred_bimod.dir_hits        3632623 # total number of direction-predicted hits (includes addr-hits)
bpred_bimod.misses           145739 # total number of misses
bpred_bimod.jr_hits          325045 # total number of address-predicted hits for JR's
bpred_bimod.jr_seen          325214 # total number of JR's seen
bpred_bimod.jr_non_ras_hits.PP         6282 # total number of address-predicted hits for non-RAS JR's
bpred_bimod.jr_non_ras_seen.PP         6324 # total number of non-RAS JR's seen
bpred_bimod.bpred_addr_rate    0.9613 # branch address-prediction rate (i.e., addr-hits/updates)
```

```
bpred_bimod.bpred_dir_rate    0.9614 # branch direction-prediction rate (i.e., all-hits/updates)
bpred_bimod.bpred_jr_rate     0.9995 # JR address-prediction rate (i.e., JR addr-hits/JRs seen)
bpred_bimod.bpred_jr_non_ras_rate.PP    0.9934 # non-RAS JR addr-pred rate (ie, non-RAS JR hits/JRs seen)
bpred_bimod.retstack_pushes       336961 # total number of address pushed onto ret-addr stack
bpred_bimod.retstack_pops         321053 # total number of address popped off of ret-addr stack
bpred_bimod.used_ras.PP           318890 # total number of RAS predictions used
bpred_bimod.ras_hits.PP           318763 # total number of RAS hits
bpred_bimod.ras_rate.PP    0.9996 # RAS prediction rate (i.e., RAS hits/used RAS)
il1.accesses                  27544052 # total number of accesses
il1.hits                      27524724 # total number of hits
il1.misses                       19328 # total number of misses
il1.replacements                 18855 # total number of replacements
il1.writebacks                       0 # total number of writebacks
il1.invalidations                    0 # total number of invalidations
il1.miss_rate                   0.0007 # miss rate (i.e., misses/ref)
il1.repl_rate                   0.0007 # replacement rate (i.e., repls/ref)
il1.wb_rate                     0.0000 # writeback rate (i.e., wrbks/ref)
il1.inv_rate                    0.0000 # invalidation rate (i.e., invs/ref)
dl1.accesses                  9068214 # total number of accesses
dl1.hits                      9025846 # total number of hits
dl1.misses                      42368 # total number of misses
dl1.replacements                41856 # total number of replacements
dl1.writebacks                  10710 # total number of writebacks
dl1.invalidations                   0 # total number of invalidations
dl1.miss_rate                   0.0047 # miss rate (i.e., misses/ref)
dl1.repl_rate                   0.0046 # replacement rate (i.e., repls/ref)
dl1.wb_rate                     0.0012 # writeback rate (i.e., wrbks/ref)
dl1.inv_rate                    0.0000 # invalidation rate (i.e., invs/ref)
ul2.accesses                    72406 # total number of accesses
ul2.hits                        62751 # total number of hits
ul2.misses                       9655 # total number of misses
ul2.replacements                 5567 # total number of replacements
ul2.writebacks                   2619 # total number of writebacks
ul2.invalidations                   0 # total number of invalidations
ul2.miss_rate                   0.1333 # miss rate (i.e., misses/ref)
ul2.repl_rate                   0.0769 # replacement rate (i.e., repls/ref)
ul2.wb_rate                     0.0362 # writeback rate (i.e., wrbks/ref)
ul2.inv_rate                    0.0000 # invalidation rate (i.e., invs/ref)
itlb.accesses                 27544052 # total number of accesses
itlb.hits                     27544032 # total number of hits
itlb.misses                         20 # total number of misses
itlb.replacements                    0 # total number of replacements
itlb.writebacks                      0 # total number of writebacks
itlb.invalidations                   0 # total number of invalidations
itlb.miss_rate                  0.0000 # miss rate (i.e., misses/ref)
```

```
itlb.repl_rate                  0.0000 # replacement rate (i.e., repls/ref)
itlb.wb_rate                    0.0000 # writeback rate (i.e., wrbks/ref)
itlb.inv_rate                   0.0000 # invalidation rate (i.e., invs/ref)
dtlb.accesses                 9101660 # total number of accesses
dtlb.hits                     9101576 # total number of hits
dtlb.misses                        84 # total number of misses
dtlb.replacements                   0 # total number of replacements
dtlb.writebacks                     0 # total number of writebacks
dtlb.invalidations                  0 # total number of invalidations
dtlb.miss_rate                  0.0000 # miss rate (i.e., misses/ref)
dtlb.repl_rate                  0.0000 # replacement rate (i.e., repls/ref)
dtlb.wb_rate                    0.0000 # writeback rate (i.e., wrbks/ref)
dtlb.inv_rate                   0.0000 # invalidation rate (i.e., invs/ref)
sim_invalid_addrs                   0 # total non-speculative bogus addresses seen (debug var)
ld_text_base           0x0120000000 # program text (code) segment base
ld_text_size                 106496 # program text (code) size in bytes
ld_data_base           0x0140000000 # program initialized data segment base
ld_data_size                  71264 # program init'ed `.data' and uninit'ed `.bss' size in bytes
ld_stack_base          0x011ff9b000 # program stack segment base (highest address in stack)
ld_stack_size                 16384 # program initial stack size
ld_prog_entry          0x01200059c0 # program entry point (initial PC)
ld_environ_base        0x011ff97000 # program environment base address address
ld_target_big_endian                0 # target executable endian-ness, non-zero if big endian
mem.page_count                     54 # total number of pages allocated
mem.page_mem                     432k # total size of memory pages allocated
mem.ptab_misses                455363 # total first level page table misses
mem.ptab_accesses           189971390 # total page table accesses
mem.ptab_miss_rate              0.0024 # first level page table miss rate
```

Compress95에 대해 Baseline.cfg로 측정한 성능은 다음과 같습니다.

```
sim: ** simulation statistics **
sim_num_insn                 88195 # total number of instructions committed
sim_num_refs                 71221 # total number of loads and stores committed
sim_num_loads                 1393 # total number of loads committed
sim_num_stores           69828.0000 # total number of stores committed
sim_num_branches              5207 # total number of branches committed
sim_elapsed_time                 1 # total simulation time in seconds
sim_inst_rate            88195.0000 # simulation speed (in insts/sec)
sim_total_insn               89265 # total number of instructions executed
sim_total_refs               71461 # total number of loads and stores executed
sim_total_loads               1559 # total number of loads executed
sim_total_stores         69902.0000 # total number of stores executed
sim_total_branches            5290 # total number of branches executed
sim_cycle                    48885 # total simulation time in cycles
sim_IPC                     1.8041 # instructions per cycle
sim_CPI                     0.5543 # cycles per instruction
sim_exec_BW                 1.8260 # total instructions (mis-spec + committed) per cycle
sim_IPB                    16.9378 # instruction per branch
IFQ_count                   170439 # cumulative IFQ occupancy
IFQ_fcount                   42270 # cumulative IFQ full count
ifq_occupancy               3.4865 # avg IFQ occupancy (insn's)
ifq_rate                    1.8260 # avg IFQ dispatch rate (insn/cycle)
ifq_latency                 1.9094 # avg IFQ occupant latency (cycle's)
ifq_full                    0.8647 # fraction of time (cycle's) IFQ was full
RUU_count                   417497 # cumulative RUU occupancy
RUU_fcount                    1859 # cumulative RUU full count
ruu_occupancy               8.5404 # avg RUU occupancy (insn's)
ruu_rate                    1.8260 # avg RUU dispatch rate (insn/cycle)
ruu_latency                 4.6771 # avg RUU occupant latency (cycle's)
ruu_full                    0.0380 # fraction of time (cycle's) RUU was full
LSQ_count                   328117 # cumulative LSQ occupancy
LSQ_fcount                   35417 # cumulative LSQ full count
lsq_occupancy               6.7120 # avg LSQ occupancy (insn's)
lsq_rate                    1.8260 # avg LSQ dispatch rate (insn/cycle)
lsq_latency                 3.6758 # avg LSQ occupant latency (cycle's)
lsq_full                    0.7245 # fraction of time (cycle's) LSQ was full
sim_slip                    895580 # total number of slip cycles
avg_sim_slip               10.1545 # the average slip between issue and retirement
bpred_bimod.lookups           5314 # total number of bpred lookups
bpred_bimod.updates           5207 # total number of updates
bpred_bimod.addr_hits         4958 # total number of address-predicted hits
bpred_bimod.dir_hits          5061 # total number of direction-predicted hits (includes addr-hits)
bpred_bimod.misses             146 # total number of misses
bpred_bimod.jr_hits             49 # total number of address-predicted hits for JR's
bpred_bimod.jr_seen            129 # total number of JR's seen
bpred_bimod.jr_non_ras_hits.PP          24 # total number of address-predicted hits for non-RAS JR's
bpred_bimod.jr_non_ras_seen.PP          66 # total number of non-RAS JR's seen
bpred_bimod.bpred_addr_rate    0.9522 # branch address-prediction rate (i.e., addr-hits/updates)
```

```
bpred_bimod.bpred_dir_rate    0.9720 # branch direction-prediction rate (i.e., all-hits/updates)
bpred_bimod.bpred_jr_rate     0.3798 # JR address-prediction rate (i.e., JR addr-hits/JRs seen)
bpred_bimod.bpred_jr_non_ras_rate.PP    0.3636 # non-RAS JR addr-pred rate (ie, non-RAS JR hits/JRs seen)
bpred_bimod.retstack_pushes          83 # total number of address pushed onto ret-addr stack
bpred_bimod.retstack_pops            70 # total number of address popped off of ret-addr stack
bpred_bimod.used_ras.PP              63 # total number of RAS predictions used
bpred_bimod.ras_hits.PP              25 # total number of RAS hits
bpred_bimod.ras_rate.PP    0.3968 # RAS prediction rate (i.e., RAS hits/used RAS)
il1.accesses                      89298 # total number of accesses
il1.hits                          89040 # total number of hits
il1.misses                          258 # total number of misses
il1.replacements                     37 # total number of replacements
il1.writebacks                        0 # total number of writebacks
il1.invalidations                     0 # total number of invalidations
il1.miss_rate                    0.0029 # miss rate (i.e., misses/ref)
il1.repl_rate                    0.0004 # replacement rate (i.e., repls/ref)
il1.wb_rate                      0.0000 # writeback rate (i.e., wrbks/ref)
il1.inv_rate                     0.0000 # invalidation rate (i.e., invs/ref)
dl1.accesses                      71268 # total number of accesses
dl1.hits                          53585 # total number of hits
dl1.misses                        17683 # total number of misses
dl1.replacements                  17171 # total number of replacements
dl1.writebacks                    17118 # total number of writebacks
dl1.invalidations                     0 # total number of invalidations
dl1.miss_rate                    0.2481 # miss rate (i.e., misses/ref)
dl1.repl_rate                    0.2409 # replacement rate (i.e., repls/ref)
dl1.wb_rate                      0.2402 # writeback rate (i.e., wrbks/ref)
dl1.inv_rate                     0.0000 # invalidation rate (i.e., invs/ref)
ul2.accesses                      35059 # total number of accesses
ul2.hits                          26030 # total number of hits
ul2.misses                         9029 # total number of misses
ul2.replacements                   4933 # total number of replacements
ul2.writebacks                     4795 # total number of writebacks
ul2.invalidations                     0 # total number of invalidations
ul2.miss_rate                    0.2575 # miss rate (i.e., misses/ref)
ul2.repl_rate                    0.1407 # replacement rate (i.e., repls/ref)
ul2.wb_rate                      0.1368 # writeback rate (i.e., wrbks/ref)
ul2.inv_rate                     0.0000 # invalidation rate (i.e., invs/ref)
itlb.accesses                     89298 # total number of accesses
itlb.hits                         89288 # total number of hits
itlb.misses                          10 # total number of misses
itlb.replacements                     0 # total number of replacements
itlb.writebacks                       0 # total number of writebacks
itlb.invalidations                    0 # total number of invalidations
itlb.miss_rate                   0.0001 # miss rate (i.e., misses/ref)
```

```
itlb.repl_rate                   0.0000 # replacement rate (i.e., repls/ref)
itlb.wb_rate                     0.0000 # writeback rate (i.e., wrbks/ref)
itlb.inv_rate                    0.0000 # invalidation rate (i.e., invs/ref)
dtlb.accesses                     71286 # total number of accesses
dtlb.hits                         71127 # total number of hits
dtlb.misses                         159 # total number of misses
dtlb.replacements                    31 # total number of replacements
dtlb.writebacks                       0 # total number of writebacks
dtlb.invalidations                    0 # total number of invalidations
dtlb.miss_rate                   0.0022 # miss rate (i.e., misses/ref)
dtlb.repl_rate                   0.0004 # replacement rate (i.e., repls/ref)
dtlb.wb_rate                     0.0000 # writeback rate (i.e., wrbks/ref)
dtlb.inv_rate                    0.0000 # invalidation rate (i.e., invs/ref)
sim_invalid_addrs                     0 # total non-speculative bogus addresses seen (debug var)
ld_text_base           0x0120000000 # program text (code) segment base
ld_text_size                     65536 # program text (code) size in bytes
ld_data_base           0x0140000000 # program initialized data segment base
ld_data_size                    743728 # program init'ed `.data' and uninit'ed `.bss' size in bytes
ld_stack_base          0x011ff9b000 # program stack segment base (highest address in stack)
ld_stack_size                    16384 # program initial stack size
ld_prog_entry          0x0120002050 # program entry point (initial PC)
ld_environ_base        0x011ff97000 # program environment base address address
ld_target_big_endian                0 # target executable endian-ness, non-zero if big endian
mem.page_count                     89 # total number of pages allocated
mem.page_mem                     712k # total size of memory pages allocated
mem.ptab_misses                   550 # total first level page table misses
mem.ptab_accesses             1515800 # total page table accesses
mem.ptab_miss_rate               0.0004 # first level page table miss rate
```

Go에 대해 Baseline.cfg로 측정한 성능은 다음과 같습니다.

```
sim: ** simulation statistics **
sim_num_insn              545812312 # total number of instructions committed
sim_num_refs              211690503 # total number of loads and stores committed
sim_num_loads             167116153 # total number of loads committed
sim_num_stores         44574350.0000 # total number of stores committed
sim_num_branches           73904242 # total number of branches committed
sim_elapsed_time                424 # total simulation time in seconds
sim_inst_rate          1287293.1887 # simulation speed (in insts/sec)
sim_total_insn            696342644 # total number of instructions executed
sim_total_refs            270206100 # total number of loads and stores executed
sim_total_loads           216505660 # total number of loads executed
sim_total_stores       53700440.0000 # total number of stores executed
sim_total_branches         92006921 # total number of branches executed
sim_cycle                 413103518 # total simulation time in cycles
sim_IPC                      1.3212 # instructions per cycle
sim_CPI                      0.7569 # cycles per instruction
sim_exec_BW                  1.6856 # total instructions (mis-spec + committed) per cycle
sim_IPB                      7.3854 # instruction per branch
IFQ_count              1025178625 # cumulative IFQ occupancy
IFQ_fcount              230361975 # cumulative IFQ full count
ifq_occupancy                2.4817 # avg IFQ occupancy (insn's)
ifq_rate                     1.6856 # avg IFQ dispatch rate (insn/cycle)
ifq_latency                  1.4722 # avg IFQ occupant latency (cycle's)
ifq_full                     0.5576 # fraction of time (cycle's) IFQ was full
RUU_count              3688147572 # cumulative RUU occupancy
RUU_fcount              111885364 # cumulative RUU full count
ruu_occupancy                8.9279 # avg RUU occupancy (insn's)
ruu_rate                     1.6856 # avg RUU dispatch rate (insn/cycle)
ruu_latency                  5.2965 # avg RUU occupant latency (cycle's)
ruu_full                     0.2708 # fraction of time (cycle's) RUU was full
LSQ_count              1442466734 # cumulative LSQ occupancy
LSQ_fcount               40279610 # cumulative LSQ full count
lsq_occupancy                3.4918 # avg LSQ occupancy (insn's)
lsq_rate                     1.6856 # avg LSQ dispatch rate (insn/cycle)
lsq_latency                  2.0715 # avg LSQ occupant latency (cycle's)
lsq_full                     0.0975 # fraction of time (cycle's) LSQ was full
sim_slip               5167480233 # total number of slip cycles
avg_sim_slip                 9.4675 # the average slip between issue and retirement
bpred_bimod.lookups        99408162 # total number of bpred lookups
bpred_bimod.updates        73904242 # total number of updates
bpred_bimod.addr_hits      57810777 # total number of address-predicted hits
bpred_bimod.dir_hits       58474173 # total number of direction-predicted hits (includes addr-hits)
bpred_bimod.misses         15430069 # total number of misses
bpred_bimod.jr_hits         6012056 # total number of address-predicted hits for JR's
bpred_bimod.jr_seen         6257514 # total number of JR's seen
bpred_bimod.jr_non_ras_hits.PP        64341 # total number of address-predicted hits for non-RAS JR's
bpred_bimod.jr_non_ras_seen.PP       117955 # total number of non-RAS JR's seen
bpred_bimod.bpred_addr_rate    0.7822 # branch address-prediction rate (i.e., addr-hits/updates)
```

```
bpred_bimod.bpred_dir_rate    0.7912 # branch direction-prediction rate (i.e., all-hits/updates)
bpred_bimod.bpred_jr_rate    0.9608 # JR address-prediction rate (i.e., JR addr-hits/JRs seen)
bpred_bimod.bpred_jr_non_ras_rate.PP    0.5455 # non-RAS JR addr-pred rate (ie, non-RAS JR hits/JRs seen)
bpred_bimod.retstack_pushes      7850560 # total number of address pushed onto ret-addr stack
bpred_bimod.retstack_pops      8846825 # total number of address popped off of ret-addr stack
bpred_bimod.used_ras.PP      6139559 # total number of RAS predictions used
bpred_bimod.ras_hits.PP      5947715 # total number of RAS hits
bpred_bimod.ras_rate.PP    0.9688 # RAS prediction rate (i.e., RAS hits/used RAS)
il1.accesses               762970475 # total number of accesses
il1.hits                   747300047 # total number of hits
il1.misses                  15670428 # total number of misses
il1.replacements            15669916 # total number of replacements
il1.writebacks                     0 # total number of writebacks
il1.invalidations                  0 # total number of invalidations
il1.miss_rate                 0.0205 # miss rate (i.e., misses/ref)
il1.repl_rate                 0.0205 # replacement rate (i.e., repls/ref)
il1.wb_rate                   0.0000 # writeback rate (i.e., wrbks/ref)
il1.inv_rate                  0.0000 # invalidation rate (i.e., invs/ref)
dl1.accesses               228444037 # total number of accesses
dl1.hits                   226896553 # total number of hits
dl1.misses                   1547484 # total number of misses
dl1.replacements             1546972 # total number of replacements
dl1.writebacks                705761 # total number of writebacks
dl1.invalidations                  0 # total number of invalidations
dl1.miss_rate                 0.0068 # miss rate (i.e., misses/ref)
dl1.repl_rate                 0.0068 # replacement rate (i.e., repls/ref)
dl1.wb_rate                   0.0031 # writeback rate (i.e., wrbks/ref)
dl1.inv_rate                  0.0000 # invalidation rate (i.e., invs/ref)
ul2.accesses                17923673 # total number of accesses
ul2.hits                    17769549 # total number of hits
ul2.misses                    154124 # total number of misses
ul2.replacements              150028 # total number of replacements
ul2.writebacks                 17455 # total number of writebacks
ul2.invalidations                  0 # total number of invalidations
ul2.miss_rate                 0.0086 # miss rate (i.e., misses/ref)
ul2.repl_rate                 0.0084 # replacement rate (i.e., repls/ref)
ul2.wb_rate                   0.0010 # writeback rate (i.e., wrbks/ref)
ul2.inv_rate                  0.0000 # invalidation rate (i.e., invs/ref)
itlb.accesses              762970475 # total number of accesses
itlb.hits                  762969330 # total number of hits
itlb.misses                     1145 # total number of misses
itlb.replacements               1081 # total number of replacements
itlb.writebacks                    0 # total number of writebacks
itlb.invalidations                 0 # total number of invalidations
itlb.miss_rate                0.0000 # miss rate (i.e., misses/ref)
```

```
itlb.repl_rate                0.0000 # replacement rate (i.e., repls/ref)
itlb.wb_rate                  0.0000 # writeback rate (i.e., wrbks/ref)
itlb.inv_rate                 0.0000 # invalidation rate (i.e., invs/ref)
dtlb.accesses              229975852 # total number of accesses
dtlb.hits                  229975170 # total number of hits
dtlb.misses                      682 # total number of misses
dtlb.replacements                560 # total number of replacements
dtlb.writebacks                    0 # total number of writebacks
dtlb.invalidations                 0 # total number of invalidations
dtlb.miss_rate                0.0000 # miss rate (i.e., misses/ref)
dtlb.repl_rate                0.0000 # replacement rate (i.e., repls/ref)
dtlb.wb_rate                  0.0000 # writeback rate (i.e., wrbks/ref)
dtlb.inv_rate                 0.0000 # invalidation rate (i.e., invs/ref)
sim_invalid_addrs                  0 # total non-speculative bogus addresses seen (debug var)
ld_text_base          0x0120000000 # program text (code) segment base
ld_text_size                376832 # program text (code) size in bytes
ld_data_base          0x0140000000 # program initialized data segment base
ld_data_size                612032 # program init'ed `.data' and uninit'ed `.bss' size in bytes
ld_stack_base         0x011ff9b000 # program stack segment base (highest address in stack)
ld_stack_size                16384 # program initial stack size
ld_prog_entry         0x0120007bb0 # program entry point (initial PC)
ld_environ_base       0x011ff97000 # program environment base address address
ld_target_big_endian               0 # target executable endian-ness, non-zero if big endian
mem.page_count                   119 # total number of pages allocated
mem.page_mem                   952k # total size of memory pages allocated
mem.ptab_misses             1938146 # total first level page table misses
mem.ptab_accesses        3857285741 # total page table accesses
mem.ptab_miss_rate            0.0005 # first level page table miss rate
```

따라서 이 내용들을 표로 정리하게 되면 다음과 같습니다.

	Gcc	Anagram	Compress95	Go
sim_num_insn	337353533	25597522	88195	545812312
sim_num_refs	121902601	9033559	71221	211690503
sim_num_loads	83209483	6491588	1393	167116153
sim_num_stores	38693118.0000	2541971.0000	69828.0000	44574350.0000
sim_num_branches	58874955	3778362	5207	73904242
sim_elapsed_time	183	10	1	424
sim_inst_rate	1843461.9290	2559752.2000	88195.0000	1287293.1887
sim_total_insn	395906021	27004989	89265	696342644
sim_total_refs	143451509	9476311	71461	270206100
sim_total_loads	100718267	6795163	1559	216505660
sim_total_stores	42733242.0000	2681148.0000	69902.0000	53700440.0000
sim_total_branches	68762165	4006787	5290	92006921
sim_cycle	273328572	11703806	48885	413103518
sim_IPC	1.2342	2.1871	1.8041	1.3212
sim_CPI	0.8102	0.4572	0.5543	0.7569
sim_exec_BW	1.4485	2.3074	1.8260	1.6856
sim_IPB	5.7300	6.7748	16.9378	7.3854
IFQ_count	621753694	41780680	170439	1025178625
IFQ_fcount	137610942	9437819	42270	230361975
ifq_occupancy	2.2747	3.5698	3.4865	2.4817
ifq_rate	1.4485	2.3074	1.8260	1.6856
ifq_latency	1.5705	1.5471	1.9094	1.4722
ifq_full	0.5035	0.8064	0.8647	0.5576
RUU_count	2301295262	160355430	417497	3688147572
RUU_fcount	66057847	6898572	1859	111885364
ruu_occupancy	8.4195	13.7011	8.5404	8.9279
ruu_rate	1.4485	2.3074	1.8260	1.6856
ruu_latency	5.8127	5.9380	4.6771	5.2965
ruu_full	0.2417	0.5894	0.0380	0.2708
LSQ_count	836129874	54506253	328117	1442466734
LSQ_fcount	30794261	1202059	35417	40279610
lsq_occupancy	3.0591	4.6571	6.7120	3.4918
lsq_rate	1.4485	2.3074	1.8260	1.6856
lsq_latency	2.1119	2.0184	3.6758	2.0715
lsq_full	0.1127	0.1027	0.7245	0.0975
sim_slip	3272483020	245007984	895580	5167480233
avg_sim_slip	9.7005	9.5716	10.1545	9.4675
bpred_bimod.lookups	72315779	4101685	5314	99408162

bpred_bimod.updates	58874955	3778362	5207	73904242
bpred_bimod.addr_hits	50991123	3632135	4958	57810777
bpred_bimod.dir_hits	52583992	3632623	5061	58474173
bpred_bimod.misses	6290963	145739	146	15430069
bpred_bimod.jr_hits	4975049	325045	49	6012056
bpred_bimod.jr_seen	6304002	325214	129	6257514
bpred_bimod.jr_non_ras_hits.PP	534855	6282	24	64341
bpred_bimod.jr_non_ras_seen.PP	1713371	6324	66	117955
bpred_bimod.bpred_addr_rate	0.8661	0.9613	0.9522	0.7822
bpred_bimod.bpred_dir_rate	0.8931	0.9614	0.9720	0.7912
bpred_bimod.bpred_jr_rate	0.7892	0.9995	0.3798	0.9608
bpred_bimod.bpred_jr_non_ras_rate.PP	0.3122	0.9934	0.3636	0.5455
bpred_bimod.retstack_pushes	5889920	336961	83	7850560
bpred_bimod.retstack_pops	5596234	321053	70	8846825
bpred_bimod.used_ras.PP	4590631	318890	63	6139559
bpred_bimod.ras_hits.PP	4440194	318763	25	5947715
bpred_bimod.ras_rate.PP	0.9672	0.9996	0.3968	0.9688
il1.accesses	427424004	27544052	89298	762970475
il1.hits	415597859	27524724	89040	747300047
il1.misses	11826145	19328	258	15670428
il1.replacements	11825633	18855	37	15669916
il1.writebacks	0	0	0	0
il1.invalidations	0	0	0	0
il1.miss_rate	0.0277	0.0007	0.0029	0.0205
il1.repl_rate	0.0277	0.0007	0.0004	0.0205
il1.wb_rate	0.0000	0.0000	0.0000	0.0000
il1.inv_rate	0.0000	0.0000	0.0000	0.0000
dl1.accesses	128845258	9068214	71268	228444037
dl1.hits	125378651	9025846	53585	226896553
dl1.misses	3466607	42368	17683	1547484
dl1.replacements	3466095	41856	17171	1546972
dl1.writebacks	902194	10710	17118	705761
dl1.invalidations	0	0	0	0
dl1.miss_rate	0.0269	0.0047	0.2481	0.0068
dl1.repl_rate	0.0269	0.0046	0.2409	0.0068
dl1.wb_rate	0.0070	0.0012	0.2402	0.0031
dl1.inv_rate	0.0000	0.0000	0.0000	0.0000
ul2.accesses	16194946	72406	35059	17923673
ul2.hits	15278731	62751	26030	17769549
ul2.misses	916215	9655	9029	154124
ul2.replacements	912119	5567	4933	150028

ul2.writebacks	208143	2619	4795	17455
ul2.invalidations	0	0	0	0
ul2.miss_rate	0.0566	0.1333	0.2575	0.0086
ul2.repl_rate	0.0563	0.0769	0.1407	0.0084
ul2.wb_rate	0.0129	0.0362	0.1368	0.0010
ul2.inv_rate	0.0000	0.0000	0.0000	0.0000
itlb.accesses	427424004	27544052	89298	762970475
itlb.hits	427307307	27544032	89288	762969330
itlb.misses	116697	20	10	1145
itlb.replacements	116633	0	0	1081
itlb.writebacks	0	0	0	0
itlb.invalidations	0	0	0	0
itlb.miss_rate	0.0003	0.0000	0.0001	0.0000
itlb.repl_rate	0.0003	0.0000	0.0000	0.0000
itlb.wb_rate	0.0000	0.0000	0.0000	0.0000
itlb.inv_rate	0.0000	0.0000	0.0000	0.0000
dtlb.accesses	129869552	9101660	71286	229975852
dtlb.hits	129822551	9101576	71127	229975170
dtlb.misses	47001	84	159	682
dtlb.replacements	46873	0	31	560
dtlb.writebacks	0	0	0	0
dtlb.invalidations	0	0	0	0
dtlb.miss_rate	0.0004	0.0000	0.0022	0.0000
dtlb.repl_rate	0.0004	0.0000	0.0004	0.0000
dtlb.wb_rate	0.0000	0.0000	0.0000	0.0000
dtlb.inv_rate	0.0000	0.0000	0.0000	0.0000
sim_invalid_addrs	0	0	0	0
ld_text_base	0x0120000000	0x0120000000	0x0120000000	0x0120000000
ld_text_size	1564672	106496	65536	376832
ld_data_base	0x0140000000	0x0140000000	0x0140000000	0x0140000000
ld_data_size	277104	71264	743728	612032
ld_stack_base	0x011ff9b000	0x011ff9b000	0x011ff9b000	0x011ff9b000
ld_stack_size	16384	16384	16384	16384
ld_prog_entry	0x0120025f70	0x01200059c0	0x0120002050	0x0120007bb0
ld_environ_base	0x011ff97000	0x011ff97000	0x011ff97000	0x011ff97000
ld_target_big_endian	0	0	0	0
mem.page_count	655	54	89	119
mem.page_mem	5240k	432k	712k	952k
mem.ptab_misses	950796	455363	550	1938146
mem.ptab_accesses	2661620259	189971390	1515800	3857285741
mem.ptab_miss_rate	0.0004	0.0024	0.0004	0.0005

저는 이번 프로젝트를 진행하면서 성능을 분석하는 지표 중 IPC와 CPI를 지표로 선택하였습니다.

	Gcc	Anagram	Compress95	Go
sim_IPC	1.2342	2.1871	1.8041	1.3212
sim_CPI	0.8102	0.4572	0.5543	0.7569

IPC와 CPI만으로 성능을 비교했을 때 Anagram이 가장 높은 IPC와 가장 낮은 CPI를 가지고 있으므로 상대적으로 가장 좋은 성능을 가지고 있으며, 순위를 매겨보면 Gcc < Go < Compress95 < Anagram 순으로 성능이 좋은 것을 알 수 있습니다.

- 시뮬레이션 결과를 토대로 Benchmark Application 특성 분석

1. Gcc Application 특성 분석

Gcc는 총 실행 시간이 183 cycle로 전체 Benchmark 중 2번째로 오래 걸리며 IPC가 1.2342로 중간 정도의 명령어 처리 속도를 보입니다. CPI가 0.8102로 안정적인 성능을 나타내고 L1I Cache와 L1D Cache의 miss rate가 0.0277, 0.0269로 중간 정도의 Cache 효율을 나타냅니다. 높은 분기 예측 정확도(0.8931)와 낮은 메모리 페이지 미스 비율(0.0004)로 안정적인 성능을 가지고 있습니다.

2. Anagram Application 특성 분석

Anagram은 총 실행 시간이 10 cycle로 전체 Benchmark 중 2번째로 짧고, IPC가 2.1871로 높은 명령어 처리 속도를 보입니다. CPI가 0.4572로 매우 효율적인 성능을 보이고 L1I Cache와 L1D Cache의 miss rate가 각각 0.0007, 0.0047로 빠른 성능을 보이고 있습니다. 높은 분기 예측 정확도(0.9614)와 우수한 메모리 성능은 단기간에 많은 작업을 처리하는 데 유용합니다.

3. Compress95 Application 특성 분석

Compress95는 총 실행 시간이 1 cycle로 전체 Benchmark 중 가장 짧고, IPC가 1.8041로 높은 명령어 처리 속도를 보입니다. CPI가 0.5543로 빠른 명령어 처리 속도를 보여주고 L1I Cache와 L1D Cache의 miss rate가 각각 0.0029, 0.2481로 Cache 효율이 낮지만, 높은 분기 예측 정확도(0.9720)와 낮은 메모리 페이지 미스 비율(0.0004)로 안정적이고 빠른 작업을 할 때 유용함을 알 수 있습니다.

4. Go Application 특성 분석

Go는 총 실행 시간이 424 cycle로 전체 Benchmark 중 가장 오래 걸리며, IPC가 1.3212로 중간 정도의 명령어 처리 속도를 가집니다. CPI가 0.7569로 안정적인 성능을 나타내고 L1I Cache와 L1D Cache의 miss rate가 각각 0.0205, 0.0068로 Cache 효율이 중간 정도이고, 높은 분기 예측 정확도(0.7912)와 낮은 메모리 페이지 미스 비율(0.0005)로 긴 실행 시간 동안 안정적인 성능을 보여줌을 알 수 있습니다.

- SimpleScalar Simulator Configuration 분석 내용

-config	설정 파일에서 설정을 로드
-dumpconfig	설정을 파일에 덤프
-h	도움말 메시지를 출력
-v	상세한 동작을 활성화
-d	디버그 메시지를 활성화
-i	Dlite 디버거에서 시작
-seed	난수 생성기 시드를 설정
-q	초기화하고 즉시 종료
-chkpt	EIO 트레이스 실행을 파일에서 복원
-redir:sim	시뮬레이터 출력을 파일로 리다이렉션함
-redir:prog	시뮬레이션된 프로그램 출력을 파일로 리다이렉션함
-nice	시뮬레이터 스케줄링 우선 순위를 설정
-max:inst	실행할 명령어의 최대 수를 설정
-fastfwd	타이밍 시작 전에 건너 뛸 명령어 수를 설정
-ptrace	파이프 트레이스를 생성하는데 사용
-fetch:ifqsize	명령어 패치 큐의 크기를 설정
-fetch:mplat	추가적인 분기 맞춤 지연(latency)을 설정
-fetch:speed	프론트엔드의 속도를 설정
-bpred	분기 예측기 유형을 설정
-bpred:bimod	이진 모달 (bimodal) 분기 예측기 구성을 설정
-bpred:2lev	2-level 분기 예측기 구성을 설정
-bpred:comb	결합 분기 예측기 구성을 설정
-bpred:ras	Return Address Stack (RAS) 크기를 설정
-bpred:btb	Branch Target Buffer (BTB) 구성을 설정
-bpred:spec_update	예측기(predictor) 업데이트를 어느 단계에서 수행할지를 설정
-decode:width	명령어 해독 대역폭을 설정
-issue:width	명령어 이슈 대역폭을 설정
-issue:inorder	명령어를 순서대로 실행
-issue:wrongpath	잘못된 실행 경로에서 명령어를 실행
-commit:width	명령어 커밋 대역폭을 설정
-ruu:size	레지스터 업데이트 유닛(RUU) 크기를 설정
-lsq:size	로드/스토어 큐(LSQ) 크기를 설정
-cache:dl1	dl1 캐시 설정
-cache:dl1lat	dl1 캐시 히트 지연 시간을 설정
-cache:dl2	dl2 캐시 설정
-cache:dl2lat	dl2 캐시 히트 지연 시간을 설정
-cache:il1	il1 캐시 설정
-cache:il1lat	il1 캐시 히트 지연 시간을 설정
-cache:il2	il2 캐시 설정
-cache:il2lat	il2 캐시 히트 지연 시간을 설정
-cache:flush	시스템 호출 시 캐시를 비움
-cache:icompress	64비트 명령어 주소를 32비트로 압축
-mem:lat	메모리 액세스 지연 시간을 설정
-mem:width	메모리 액세스 버스 폭을 설정
-tlb:itlb	명령어(코드)를 번역하는 데 사용되는 명령어 tlb의 구성을 설정

-tlb:dtlb	데이터를 번역하는 데 사용되는 데이터 tlb의 구성을 설정
-tlb:lat	tlb 미스 지연 시간을 설정
-res:ialu	사용 가능한 정수 ALU(산술 논리 장치)의 총 수를 설정
-res:imult	사용 가능한 정수 곱셈기/나눗셈기의 총 수를 설정
-res:memport	사용 가능한 메모리 시스템 포트의 총 수를 설정
-res:fpalu	사용 가능한 부동 소수점 ALU(산술 논리 장치)의 총 수를 설정
-res:fpmult	사용 가능한 부동 소수점 곱셈기/나눗셈기의 총 수를 설정
-pcstat	텍스트 주소에 대한 프로파일 통계를 생성
-bugcompat	역방향 호환성 버그 모드에서 실행

여기서 Important Configurations는 다음과 같습니다.

-fetch:ifqsize	명령어 패치 큐의 크기를 설정
-bpred	분기 예측기 유형을 설정
-decode:width	명령어 해독 대역폭을 설정
-issue:width	명령어 이슈 대역폭을 설정
-cache:dl1	dl1 캐시 설정
-cache:dl2	dl2 캐시 설정
-cache:il1	il1 캐시 설정
-cache:il2	il2 캐시 설정
-res:ialu	사용 가능한 정수 ALU(산술 논리 장치)의 총 수를 설정
-res:imult	사용 가능한 정수 곱셈기/나눗셈기의 총 수를 설정
-res:memport	사용 가능한 메모리 시스템 포트의 총 수를 설정
-res:fpalu	사용 가능한 부동 소수점 ALU(산술 논리 장치)의 총 수를 설정
-res:fpmult	사용 가능한 부동 소수점 곱셈기/나눗셈기의 총 수를 설정

성능을 올리기 위해 Important Configurations 이외에 추가로 변경한 Configurations로는 아래와 같은 항목들이 있습니다.

-commit:width	명령어 커밋 대역폭을 설정
-ruu:size	레지스터 업데이트 유닛(RUU) 크기를 설정
-lsq:size	로드/스토어 큐(LSQ) 크기를 설정
-mem:width	메모리 액세스 버스 폭을 설정
-tlb:itlb	명령어(코드)를 번역하는 데 사용되는 명령어 tlb의 구성을 설정
-tlb:dtlb	데이터를 번역하는 데 사용되는 데이터 tlb의 구성을 설정

1. Gcc Benchmark

- Baseline.cfg 대비 변경된 Configuration

	Baseline.cfg	변경된 Configuration
-fetch:ifqsize	4	32
-bpred	bimod	perfect
-decode:width	4	32
-issue:width	4	32
-cache:dl1	dl1:128:32:4:l	dl1:4096:256:8:l
-cache:dl2	ul2:1024:64:4:l	ul2:8192:236:8:l
-cache:il1	il1:512:32:1:l	il1:4096:256:8:l
-cache:il2	dl2	dl2
-res:ialu	4	8
-res:imult	1	8
-res:memport	2	8
-res:fpalu	4	8
-res:fpmult	1	8
-commit:width	4	128
-ruu:size	16	256
-lsq:size	8	256
-mem:width	8	16
-tlb:itlb	itlb:16:4096:4:l	itlb:64:4096:16:l
-tlb:dtlb	dtlb:32:4096:4:l	dtlb:64:4096:16:l

- Configuration 변경 이유

fetch:ifqsize를 변경하여 한번에 더 많은 명령어를 가져와 decoding할 수 있도록 하여 명령어 처리 효율을 향상시키려 하였습니다. decode:width와 issue:width을 늘려 명령어 처리 효율을 향상시키려고 하였는데 여러 번 테스트 해본 결과 64, 128로 늘렸을 때보다 32가 IPC와 CPI의 성능이 가장 높아지는 것을 알 수 있었습니다. cache:dl1과 cache:dl2를 늘려 cache 히트율을 높이고 메모리에 접근하는 시간을 단축시키려고 하였다. 또한 cache:il1의 블록 크기와 연관성을 늘려서 cache의 충돌을 줄일 수 있도록 하고 명령어를 더 빨리 가져올 수 있도록 하였습니다.

res:ialu, res:imult, res:memport, res:fpalu, res:fpmult의 수를 증가시켜서 한번에 더 많은 연산들을 동시에 처리할 수 있도록 만들려고 하였습니다.

bpred를 bimod에서 perfect로 변경하여 실제 분기 명령의 결과를 정확하게 예측시키고,

commit:width을 늘려서 한번에 많은 명령어를 commit할 수 있게 하여 처리량을 향상시키려고 하였습니다.

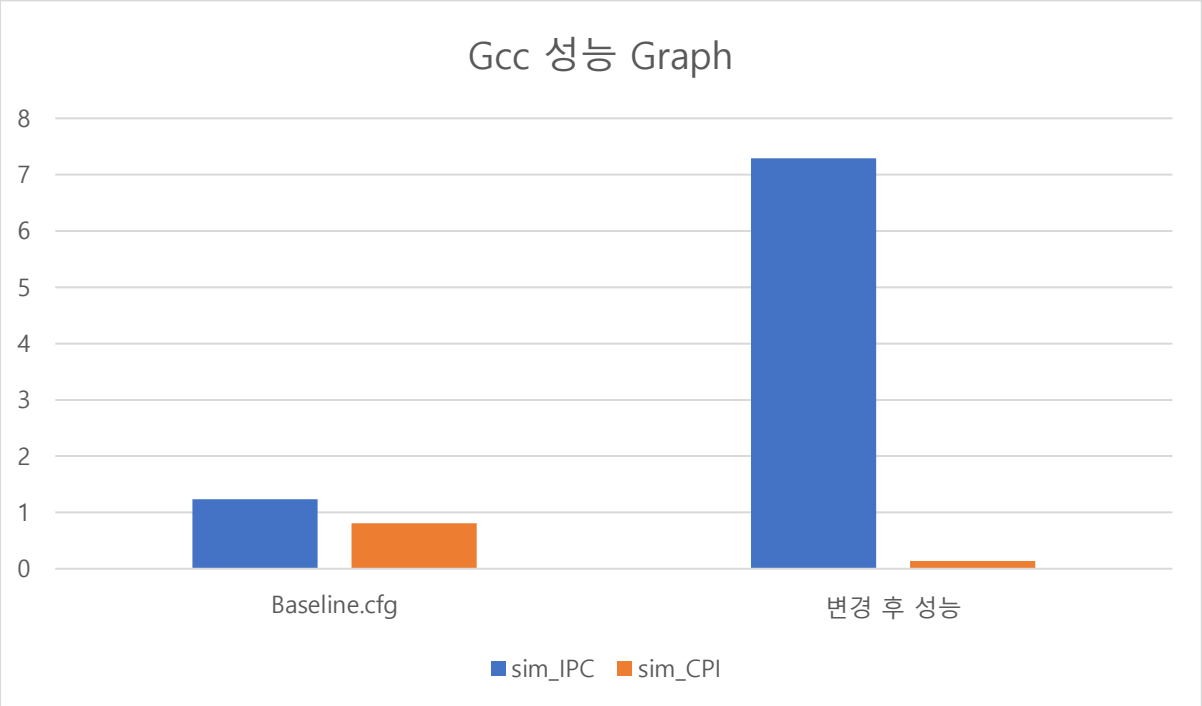
ruu와 lsq의 size를 늘려서 명령어 병렬 처리 능력을 향상시키려 하였고 Cache Miss나 다른 예측 실패로 인한 지연이 발생했을 때 프로세서가 대기하지 않고 다른 명령어를 수행할 수 있도록 하였습니다.

mem:width을 늘려 메모리 인터페이스의 대역폭을 늘려서 메모리에 접근할 수 있는 접근속도를 향상시켜서 프로세서의 성능을 높이려 하였고, tlb:itlb와 tlb:dtlb로 tlb의 크기를 늘려 메모리에 접근하는 시간을 줄이려 하였습니다.

- Configuration 적용 시 주요 성능 Graph 및 분석

	Baseline.cfg	**변경 후 성능**
sim_IPC	1.2342	7.2885
sim_CPI	0.8102	0.1372

위와 같은 Configuration 변경들로 IPC와 CPI를 분석해본 결과 약 5.9배 정도 성능이 오른 것을 알 수 있었습니다.



위 내용과 같은 Configuration 변경들로 시스템의 병렬 처리 능력을 향상시켰고, 메모리에 접근하는 시간을 줄여 IPC와 CPI의 성능이 약 5.9배 정도 향상되었습니다.

2. Anagram Benchmark

- Baseline.cfg 대비 변경된 Configuration

	Baseline.cfg	변경된 Configuration
-fetch:ifqsize	4	256
-bpred	bimod	perfect
-decode:width	4	64
-issue:width	4	64
-cache:dl1	dl1:128:32:4:l	dl1:1024:128:4:l
-cache:dl2	ul2:1024:64:4:l	ul2:2048:128:4:l
-cache:il1	il1:512:32:1:l	il1:1024:128:4:l
-cache:il2	dl2	dl2
-res:ialu	4	8
-res:imult	1	8
-res:memport	2	8
-res:fpalu	4	8
-res:fpmult	1	8
-commit:width	4	128
-ruu:size	16	512
-lsq:size	8	512
-mem:width	8	16
-tlb:itlb	itlb:16:4096:4:l	itlb:64:4096:8:l
-tlb:dtlb	dtlb:32:4096:4:l	dtlb:64:4096:8:l

- Configuration 변경 이유

fetch:ifqsize를 변경하여 한번에 더 많은 명령어를 가져와 decoding할 수 있도록 하여 전체적인 명령어 처리 효율을 향상시키고, decode:width와 issue:width을 늘려 명령어 처리 효율을 향상시키려고 하였습니다. cache:dl1과 cache:dl2를 늘려 cache 히트율을 높이고 메모리에 접근하는 시간을 단축시키려고 하였습니다. 또한 cache:il1의 블록 크기와 연관성을 늘려서 cache의 충돌을 줄일 수 있도록 하고 명령어를 더 빨리 가져올 수 있도록 하였습니다.

res:ialu, res:imult, res:memport, res:fpalu, res:fpmult의 수를 증가시켜서 한번에 더 많은 연산들을 동시에 처리할 수 있도록 만들려고 하였습니다.

bpred를 bimod에서 perfect로 변경하여 실제 분기 명령의 결과를 정확하게 예측시키고, commit:width을 늘려서 한번에 많은 명령어를 commit할 수 있게 하여 처리량을 향상시키려고 하

였습니다.

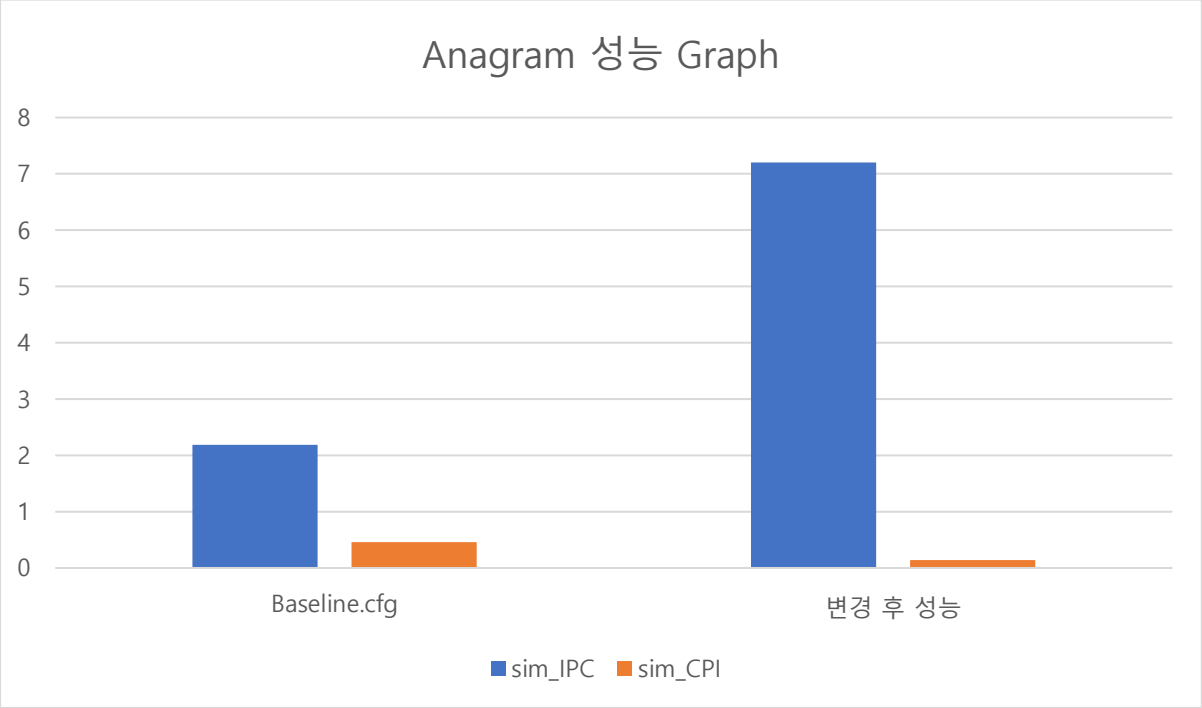
ruu와 lsq의 size를 늘려서 명령어 병렬 처리 능력을 향상시키려 하였고 Cache Miss나 다른 예측 실패로 인한 지연이 발생했을 때 프로세서가 대기하지 않고 다른 명령어를 수행할 수 있도록 하였습니다.

mem:width을 늘려 메모리 인터페이스의 대역폭을 늘려서 메모리에 접근할 수 있는 접근속도를 향상시키려 하였고, tlb:itlb와 tlb:dtlb로 tlb의 크기를 늘려 메모리에 접근하는 시간을 줄이려 하였습니다.

- Configuration 적용 시 주요 성능 Graph 및 분석

	Baseline.cfg	**변경 후 성능**
sim_IPC	2.1871	7.1984
sim_CPI	0.4572	0.1389

위와 같은 Configuration 변경들로 IPC와 CPI를 분석해본 결과 IPC와 CPI모두 약 3.29배 정도 성능이 오른 것을 알 수 있었습니다.



위 내용과 같은 Configuration 변경들로 프로세서가 주어진 주기 동안 더 많은 명령어를 처리하며, 시스템의 병렬 처리 능력을 향상시켰고, 메모리에 접근하는 시간을 줄여 시스템의 성능이 약 3.29배 정도 향상되었습니다.

3. Compress95 Benchmark

- Baseline.cfg 대비 변경된 Configuration

	Baseline.cfg	변경된 Configuration
-fetch:ifqsize	4	256
-bpred	bimod	perfect
-decode:width	4	16
-issue:width	4	16
-cache:dl1	dl1:128:32:4:l	dl1:1024:128:4:l
-cache:dl2	ul2:1024:64:4:l	ul2:2048:128:4:l
-cache:il1	il1:512:32:1:l	il1:1024:128:4:l
-cache:il2	dl2	dl2
-res:ialu	4	8
-res:imult	1	8
-res:memport	2	8
-res:fpalu	4	8
-res:fpmult	1	8
-commit:width	4	128
-ruu:size	16	1024
-lsq:size	8	1024
-mem:width	8	16
-tlb:itlb	itlb:16:4096:4:l	itlb:64:4096:8:l
-tlb:dtlb	dtlb:32:4096:4:l	dtlb:64:4096:8:l

- Configuration 변경 이유

fetch:ifqsize를 변경하여 한번에 더 많은 명령어를 가져와 decoding할 수 있도록 하여 명령어 처리 효율을 향상시키고 전체적인 처리량이 향상될 수 있습니다. 또한 decode:width와 issue:width을 늘려 명령어 처리 효율을 향상시키려고 하였습니다. cache:dl1과 cache:dl2를 늘려 cache 히트율을 높이고 메모리에 접근하는 시간을 단축시키려고 하였습니다. cache:il1의 블록 크기와 연관성을 늘려서 cache의 충돌을 줄일 수 있도록 하고 명령어를 더 빨리 가져올 수 있도록 하였습니다.

res:ialu, res:imult, res:memport, res:fpalu, res:fpmult의 수를 증가시켜서 한번에 더 많은 연산들을 동시에 처리할 수 있도록 만들려고 하였습니다.

bpred를 bimod에서 perfect로 변경하여 실제 분기 명령의 결과를 정확하게 예측시키고, commit:width을 늘려서 한번에 많은 명령어를 commit할 수 있게 하여 처리량을 향상시키려고 하

였습니다.

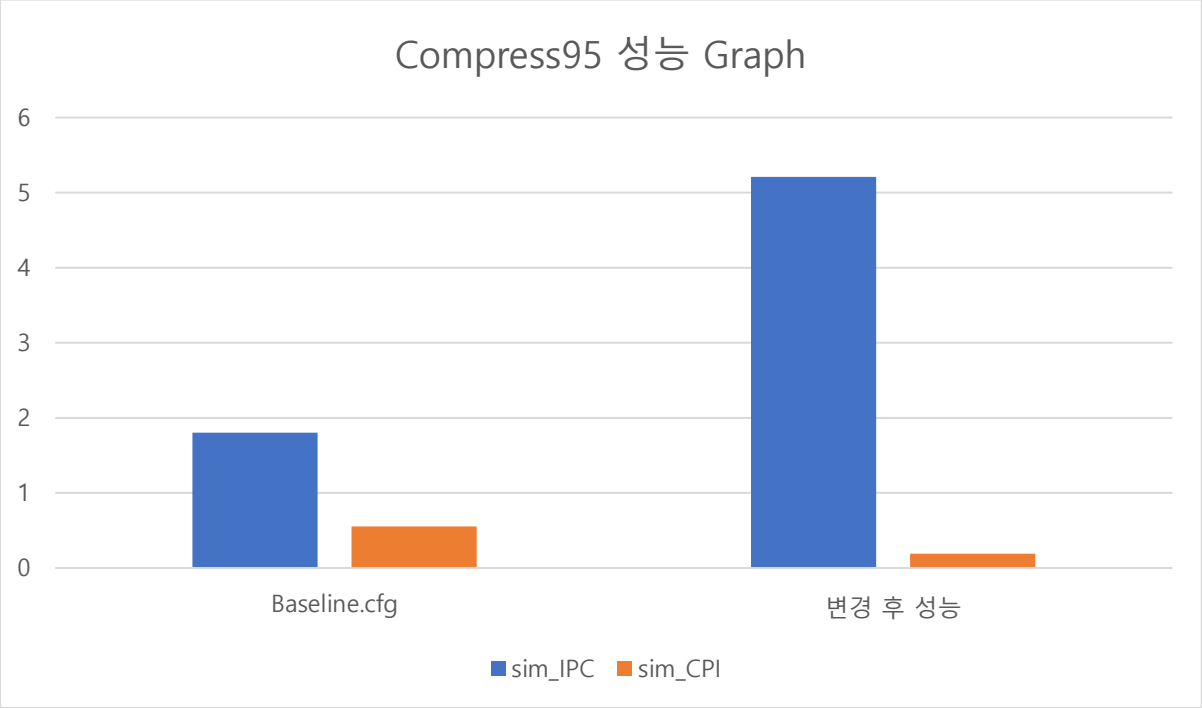
ruu와 lsq의 size를 늘려서 명령어 병렬 처리 능력을 향상시키려 하였고 Cache Miss나 다른 예측 실패로 인한 지연이 발생했을 때 프로세서가 대기하지 않고 다른 명령어를 수행할 수 있도록 하였습니다.

mem:width을 늘려 메모리 인터페이스의 대역폭을 늘려서 메모리에 접근할 수 있는 접근속도를 향상시키려 하였고, tlb:itlb와 tlb:dtlb로 tlb의 크기를 늘려 메모리에 접근하는 시간을 줄여 메모리 access를 빠르게 가능하도록 하였습니다.

- Configuration 적용 시 주요 성능 Graph 및 분석

	Baseline.cfg	**변경 후 성능**
sim_IPC	1.8041	5.2091
sim_CPI	0.5543	0.1920

위와 같은 Configuration 변경들로 IPC와 CPI를 분석해본 결과 IPC와 CPI모두 약 2.89배 정도 성능이 오른 것을 알 수 있었습니다.



위 내용과 같은 Configuration 변경들로 프로세서가 주어진 주기 동안 더 많은 명령어를 처리하며, 시스템의 병렬 처리 능력을 향상시켰고, 메모리에 접근하는 시간을 줄여 시스템의 성능이 약 2.89배 정도 향상되었습니다.

4. Go Benchmark

- Baseline.cfg 대비 변경된 Configuration

	Baseline.cfg	변경된 Configuration
-fetch:ifqsize	4	256
-bpred	bimod	perfect
-decode:width	4	32
-issue:width	4	32
-cache:dl1	dl1:128:32:4:l	dl1:1024:64:4:l
-cache:dl2	ul2:1024:64:4:l	ul2:2048:64:4:l
-cache:il1	il1:512:32:1:l	il1:1024:64:4:l
-cache:il2	dl2	dl2
-res:ialu	4	8
-res:imult	1	8
-res:memport	2	8
-res:fpalu	4	8
-res:fpmult	1	8
-commit:width	4	128
-ruu:size	16	512
-lsq:size	8	512
-mem:width	8	16
-tlb:itlb	itlb:16:4096:4:l	itlb:64:4096:8:l
-tlb:dtlb	dtlb:32:4096:4:l	dtlb:64:4096:8:l

- Configuration 변경 이유

fetch:ifqsize를 변경하여 한번에 더 많은 명령어를 가져와 decoding할 수 있도록 하여 명령어 처리 효율을 향상시키고, decode:width와 issue:width을 늘려 명령어 처리 효율을 향상시키려고 하였습니다. cache:dl1과 cache:dl2를 늘려 cache 히트율을 높이고 메모리에 접근하는 시간을 단축시키려고 하였습니다. 또한 cache:il1의 블록 크기와 연관성을 늘려서 cache의 충돌을 줄일 수 있도록 하고 명령어를 더 빨리 가져올 수 있도록 하였습니다.

res:ialu, res:imult, res:memport, res:fpalu, res:fpmult의 수를 증가시켜서 한번에 더 많은 연산들을 동시에 처리할 수 있도록 만들려고 하였습니다.

bpred를 bimod에서 perfect로 변경하여 실제 분기 명령의 결과를 정확하게 예측시키고, commit:width을 늘려서 한번에 많은 명령어를 commit할 수 있게 하여 처리량을 향상시키려고 하

였습니다.

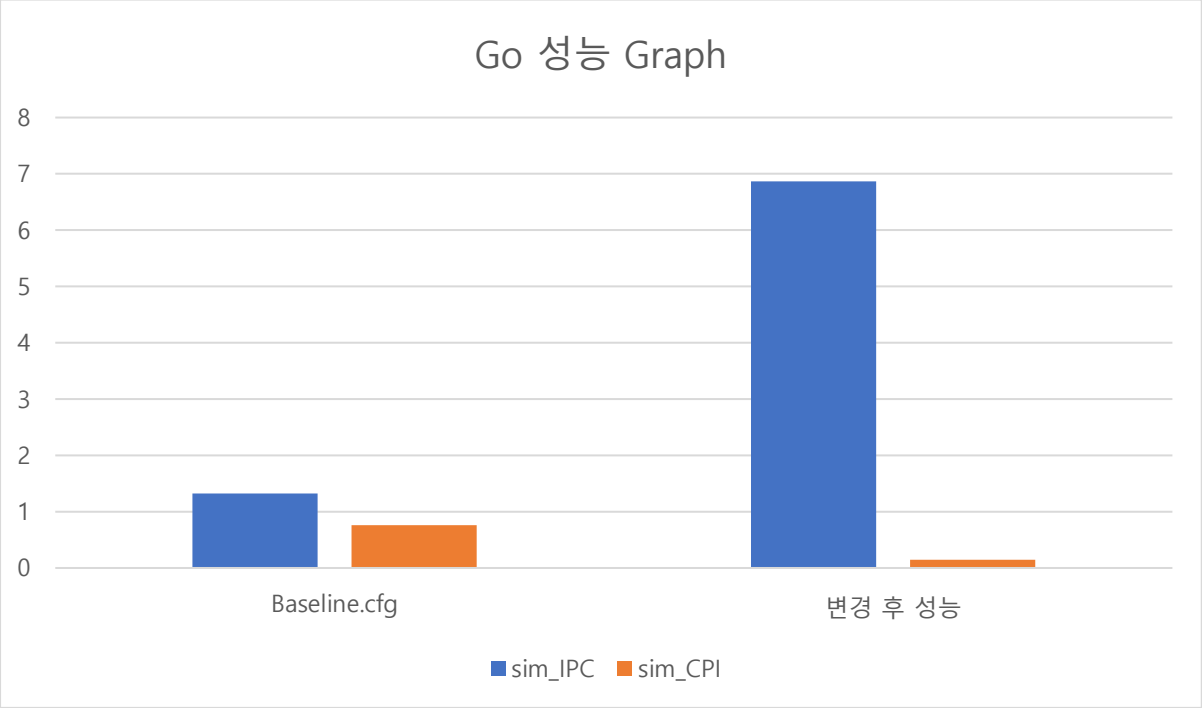
ruu와 lsq의 size를 늘려서 명령어 병렬 처리 능력을 향상시키려 하였고 Cache Miss나 다른 예측 실패로 인한 지연이 발생했을 때 프로세서가 대기하지 않고 다른 명령어를 수행할 수 있도록 하였습니다.

mem:width을 늘려 메모리 인터페이스의 대역폭을 늘려서 메모리에 접근할 수 있는 접근속도를 향상시키려 하였고, tlb:itlb와 tlb:dtlb로 tlb의 크기를 늘려 메모리에 접근하는 시간을 줄이려 하였습니다.

- Configuration 적용 시 주요 성능 Graph 및 분석

	Baseline.cfg	**변경 후 성능**
sim_IPC	1.3212	6.8668
sim_CPI	0.7569	0.1456

위와 같은 Configuration 변경들로 IPC와 CPI를 분석해본 결과 약 5.2배 정도 성능이 오른 것을 알 수 있었습니다.



위 내용과 같은 Configuration 변경들로 프로세서가 주어진 주기 동안 더 많은 명령어를 처리하며, 시스템의 병렬 처리 능력을 향상시켰고, 그 결과 시스템의 성능이 약 5.2배 정도 향상되었습니다.